

DeskIt (DSKT)

Implementation Plan & Progress Report

Office Workspace Mapping & Seating Management Platform
Generated: 29 September 2026 | Status: Prototype (frontend-complete, backend pending)

1. Executive Summary

DeskIt is a substantial two-frontend prototype. The floor mapper (Creator) and role-based seating UX (DeskIt frontend) are largely built on browser localStorage and mock data. Production backend, real SSO, and durable history remain the main remaining work.

Core principle: the mapper creates a logical spatial model; the backend persists that model; the viewer renders that model.

2. What DeskIt Is

- Three intended layers:
- Mapper (Creator) - Admins design floor geometry as FloorDocument JSON.
 - Viewer / Manager (DeskIt frontend) - Employees, HR, and Admins view maps, assign seats, find people.
 - Backend (planned) - Persist the logical model (Spring Boot + PostgreSQL per handoff).

Item	Detail
Floor Creator	creator/grid-ui - React 19, TypeScript, Vite, SVG, Vitest, jsPDF
DeskIt app	frontend - React 18, TypeScript, Vite, Tailwind, Lucide, optional Supabase
Assets	assets/2d + assets/catalog - SVG furniture / structure library
Persistence today	Browser localStorage + mock data (no server of record)
Auth today	Demo SSO modal + role switcher (not real SSO)

3. What We Have Done

A. Floor Creator / Mapper (largely complete)

CAD-like planner with a hierarchical grid - the mathematical foundation of the product.

Spatial model:

- First-quadrant Y-up world; SVG screen space with Y flipped.
- Fixed grid levels: 2a -> a -> a/4 -> a/16 (storage at a/16).
- Cursor-anchored zoom, pan clamping, adaptive grid density, Fit.

Editing:

- Catalog library with SVG previews; place ghost; snap to placement grid.
- Place / move / resize / 90-degree rotate; hit-test; custom polygons.
- Named zones; unusable regions that block overlap; undo/redo; layout scale.

Persistence & export:

- FloorDocument v2 JSON (entities, zones, custom library, unusable, viewport, theme).
- Drafts in localStorage (up to 40); JSON import/export; PNG / SVG / PDF export.
- Import walls/rooms from a floor-plan image.

Integration:

- Publish bridge into DeskIt via localStorage + postMessage iframe contract.

- Geometry unit tests under creator/grid-ui.

B. DeskIt Frontend - Role-Based Product Shell (mock-backed)

Auth & roles (demo): employee | hr | admin; permission matrix sketched in lib/permissions.ts.

Employee: Home, Floor Maps, Find People; published SVG map viewer; presence colors.

HR: Overview, People & Teams, Seat Allocation, Floor Maps, Seat Requests; permanent/temporary assign; area-select to team; floor change requests; ~999 mock employees.

Admin: Overview, Floor Plan Editor (Creator embed), Drafts & Versions, Change Requests, Published Maps; publish / clone / new floors; version badges.

Org model (mock): Organization -> Office -> Floor -> FloorPlan/FloorDocument -> Elements -> Seats; Employee -> Team -> SeatAssignment.

C. What 'Done' Means Today

Rich UX and geometry work as a browser prototype. There is no real server of record. Persistence is localStorage + mocks. Auth is not production SSO.

4. Implementation Plan - Completed vs Future

Phase 0 - Product Foundation (DONE)

- ☒ Product vision & AI handoff specification
- ☒ Two-frontend split: Mapper vs Viewer/Manager
- ☒ Shared mental model: logical floor document as source of truth
- ☒ SVG asset catalog

Phase 1 - Mapper Prototype / Milestone 1 (MOSTLY DONE)

Criterion	Status
Hierarchical grid + unit a	Done (fixed 4 levels)
Zoom / pan / fit, cursor-anchored	Done
Place / move / resize / 90 rotate / snap	Done
Zones / areas / custom shapes	Done
Undo/redo + serialize/deserialize	Done
Wall drawing as polylines	Partial
Konva + Zustand (per handoff)	Not used (SVG + hooks)

Phase 2 - Publish Bridge Creator <-> DeskIt (DONE in prototype)

- ☒ FloorDocument v2 as interchange format
- ☒ Embed Creator in Admin
- ☒ Publish -> live map for HR/Employee
- ☒ Drafts vs live version badges (client-side)

Phase 3 - Role UX & Seating Workflows (DONE as mocks)

- ☒ Employee / HR / Admin shells and permissions
- ☒ Seat assign UI, team area assign, people directory, find people
- ☒ Floor change requests (local)
- ☒ Demo SSO + role switch

Phase 4 - Production Persistence & Auth (FUTURE - highest priority)

- ☐ Backend API (Java / Spring Boot / Security / JPA)
- ☐ PostgreSQL schema for floors, documents, employees, teams, assignments, requests
- ☐ Replace localStorage as system of record
- ☐ Real SSO; server-side role enforcement
- ☐ Commit to Spring or finish/retire optional Supabase path

Phase 5 - Durable Business Workflows (FUTURE)

- ☐ Seat assignment requests with durable approve/reject
- ☐ Floor change requests end-to-end with audit
- ☐ MovementHistory (assign / unassign / move over time)
- ☐ Full floor-plan version history + restore
- ☐ Deepen HR assign flows

Phase 6 - Geometry & Mapper Polish (FUTURE)

- ☐ Proper wall / door / opening model
- ☐ Decide: fixed grid levels vs configurable hierarchy
- ☐ Real-world units (meters/feet) if needed
- ☐ Retire or rewire orphaned FloorPlanEditor.tsx
- ☐ Harden cross-origin Creator <-> DeskIt for production hosting

Phase 7 - Product Hardening (FUTURE)

- ☐ URL routing / deep links (floor, tab, employee)
- ☐ React Query (or equivalent) for server state
- ☐ Performance for large floors; optional spatial indexing
- ☐ Object storage for large exports/assets if needed
- ☐ Proper READMEs and a single living roadmap
- ☐ Align handoff tech (Konva/Zustand) with shipped SVG - or update the spec

5. Suggested Near-Term Sequence

1. Freeze FloorDocument v2 as the API contract
2. Build Spring + PostgreSQL (or commit to Supabase) for floors + documents
3. Wire real SSO + roles (reuse frontend permission matrix)
4. Persist seat assignments & requests; add MovementHistory
5. Version history + restore for published plans
6. Wall/door geometry + cleanup of legacy editors
7. Routing, performance, operations, documentation

6. Architecture - Today vs Target

TODAY		TARGET
-----		-----
Creator (SVG editor)		Creator (same or evolved)
postMessage / localStorage		HTTPS API
v		v
DeskIt frontend (roles, UX)	->	DeskIt frontend
localStorage / mocks		
v		v
(no server)		Spring Boot + PostgreSQL
		(+ real SSO)

7. Gaps & Tradeoffs

- Spec drift: Handoff assumed Konva + Zustand + configurable grid levels; shipped mapper is SVG with a fixed level ladder.
- Business layer is UI-complete, data-incomplete: HR/Admin flows look real but reset with browser storage.
- Two editors: Creator is the path forward; leftover FloorPlanEditor should not be extended.
- Milestone 1 walls: Still the biggest mapper gap vs original success criteria.

8. Progress Snapshot

Area	Progress
Spatial mapper + FloorDocument	~90% for a prototype; walls/config levels left
Publish / view / role UX	Solid client prototype
Seating / people / requests	Implemented against mocks
Backend / DB / real auth / history	Not started (or only Supabase stubs)

9. Bottom Line

DONE: Mathematical floor planning, catalog editing, and publish into a multi-role DeskIt application.

NEXT: Make that model and those workflows real with a backend, authentication, and durable seating/version history.

Sources: creator/document.md, creator/grid-ui/FEATURES.txt, features.md, frontend/src/lib/dataModel.ts, and Workspace Management System - Complete AI Handoff Specification.md. This document is informational; no application code was modified to produce it.